

MCP サーバ仕様

目次

1 登録(対象プロジェクトの .mcp.json)	2
2 MCP 化しないもの(意図的な除外)	2
3 台帳(読み取り) — ledger.py	3
4 ④ 機械抽出 — extract_fortran.py / extract_c.py	3
5 機械レビュー — review_checks.py	3
6 台帳(書き込み) — ledger.py	4
7 実行系	5
8 出力系	5

mcp-servers/legacy-reverse-mcp/server.py(FastMCP)は、skills が Bash 経由で叩いていた スクリプト層を型付きツールとして公開するサーバ。

- 判断は持たない。実体は legacy-reverse/scripts/ の各スクリプトを import または subprocess で呼ぶだけ(単体でも従来どおり動く。二重管理なし)
- 戻り値は構造化(dict / list)。シェル引用の事故と許可プロンプトを減らす
- 登録は推奨だが必須ではない。未登録環境では skill がスクリプトを直接実行する (workflow.md の規則: 登録済み環境では MCP ツール優先)
- 全ツールが root(対象プロジェクトのルート、既定 ".")を取る

1 登録(対象プロジェクトの .mcp.json)

```
{
  "mcpServers": {
    "legacy-reverse": {
      "command": "python",
      "args": ["<skills リポジトリ>/mcp-servers/legacy-reverse-mcp/server.py"]
    }
  }
}
```

2 MCP 化しないもの(意図的な除外)

表 2-1

対象	理由
serve_site.py / browser_run.py / review_actions.py	常駐プロセスであり、かつ人の操作の入口(実行・承認・裁定の API)。LLM に渡すツールではない
hooks/guard_tests.py・guard_json.py	ツール呼び出しの強制ガードは hook の役割のまま (MCP 化すると迂回可能になる)

3 台帳(読み取り) — ledger.py

表 3-1

ツール	引数	返り値	説明
<code>pipeline_status</code>	<code>root, func_id?</code>	<code>list[dict]</code>	全関数(または指定関数)の①～⑤の進捗・stale・blocked 状態。WBS と同じ判定ロジック(<code>status_of()</code>)の生データ
<code>next_action</code>	<code>root</code>	<code>str</code>	トポロジカル順(葉から)で次に着手すべき関数とフェーズ
<code>next_actions</code>	<code>root, limit=20, phase?, skip_draft=False</code>	<code>str</code>	着手可能な関数を推奨順に最大 limit 件 (<code>fid<TAB>次フェーズ</code>)。バッチ計画用。 <code>phase="1"~"5"</code> で絞り込み、 <code>skip_draft=True</code> で人のレビュー待ち(① draft・② generated)を除外 — バッチ全件モードの再開時に使う(draft の二重書き直し防止)
<code>progress_summary</code>	<code>root</code>	<code>dict</code>	各フェーズの完了数・blocked・stale・改変・fail の要約 JSON。2000 関数規模でも全関数リストを読まずに状況把握できる
<code>verify</code>	<code>root, func_id</code>	<code>dict</code>	ハッシュ連鎖(①→②、③改変、blocked)の検証。 <code>ok=false</code> なら <code>problems</code> に理由
<code>next_issue_id</code>	<code>root</code>	<code>str</code>	次に使う ISSUE 番号(全体通し)

4 ④ 機械抽出 — `extract_fortran.py` / `extract_c.py`

表 4-1

ツール	引数	返り値	説明
<code>extract_functions</code>	<code>root, legacy_dir="legacy", package?, write=True, infer_calls=True</code>	<code>dict</code>	Fortran ソースを静的解析し <code>functions.json</code> を生成/マージ。再実行は常にマージ(<code>func_id</code> 不変・手修正保持)。返り値に完全性突合の差分・推定呼び出し・未解決名の件数。詳細は <code>data/extract-report.json</code>
<code>extract_c_functions</code>	<code>root, legacy_dir="legacy", package?, write=True, infer_calls=True</code>	<code>dict</code>	C/C++ を同じ <code>functions.json</code> にマージ。Fortran↔C の呼び出しをアンダースコア規約込みで突合するので、抽出の実行順は問わない(後から走った側が未解決名を解決する)。監査ログは <code>data/extract-report-c.json</code>

5 機械レビュー — `review_checks.py`

表 5-1

ツール	引数	返り値	説明
<code>review_spec</code>	root, func_id	dict	①仕様書の機械レビュー。① draft 後・人レビュー依頼前に必ず実行し ok=true にしてから人に出す
<code>review_testspec</code>	root, func_id	dict	②テスト仕様書の機械レビュー。approved 依頼前に必ず実行
<code>review_all</code>	root	dict	全関数の①②一括レビュー(skeleton 除外)。⑥前の総点検
<code>spec_review_report</code>	root	dict	① draft の一斉レビュー表(docs/spec-review.md)を生成。生成後に render_site で HTML 化して人に案内する

検査内容の詳細は 品質ゲート仕様。

6 台帳(書き込み) — ledger.py

表 6-1

ツール	引数	返り値	説明
<code>add_function</code>	root, legacy_file, legacy_name, new_module?, new_name?, lines?, note?	dict	⑤の抽出漏れ・関数分割を後追いで追加。 manual: true が付くので再抽出で「ソースに無い」警告が出ない (実ソースで確認されたら自動で外れる)
<code>exclude_function</code>	root, func_id, reason	dict	移植対象外にする(デッドコード等)。①～⑥・WBS・next の対象から外れ、WBS の「対象外の関数」に 理由つきで残る。物理削除はしない (再抽出で別 ID として復活し成果物との紐付けが切れるため)
<code>include_function</code>	root, func_id	dict	除外を取り消して対象に戻す
<code>generate_wbs</code>	root	dict	functions.json+成果物フロントマターから docs/index.qmd(WBS)を再生成
<code>generate_skeletons</code>	root, force=False	dict	functions.json から仕様書骨子を生成(既存はスキップ)
<code>freeze_tests</code>	root, func_id	dict	③完了時にテストコードのハッシュを台帳へ記録(以降の改変を検知可能に)
<code>block</code>	root, func_id, issue_id	dict	関数を裁定待ち(🔴)にする
<code>unblock</code>	root, func_id	dict	人の裁定反映後にブロック解除し attempt をリセット
<code>phase_start</code>	root, phase, func_id	dict	フェーズ開始を記録(4/5の間は hook が tests/ 編集を拒否)
<code>phase_end</code>	root	dict	フェーズ終了を記録(hook 解除)
<code>completion_check</code>	root	dict	⑥完了検証を実行し docs/completion-check.md を生成。ok=true が⑥ pass

ツール	引数	返り値	説明
<code>sphinx_index</code>	root	dict	functions.json から docs-sphinx/index.rst(関数一覧+トレース対応表)を再生成

7 実行系

表 7-1

ツール	引数	返り値	説明
<code>run_tests</code>	root, func_id	dict	⑤の一括実行: 事前 verify → pytest(tc マーカー収集)→ 結果報告書の自動生成。result: pass / fail / blocked(要裁定) / mismatch(③マーカー不整合) / verify_ng。報告書パス・実装率(rate)・attempt・blocked_by も返す
<code>check_stubs</code>	root, module	dict	④のスタブ検出。ok=true でスタブなし
<code>profile</code>	root, script?	dict	⑦の性能計測(cProfile → docs/perf.md)。script 未指定はテスト負荷のスモーク計測。ホットスポット判断は代表ワークロードで

8 出力系

表 8-1

ツール	引数	返り値	説明
<code>render_site</code>	root, with_sphinx=True, full=False	dict	docs/ を Quarto で HTML 化 → 続けて Sphinx API を docs/_site/api に生成(正順を内包)。quarto 直叩きせず render_site.py を通す(Mermaid 影コピーのため)。既定は差分レンダ(変わったページだけ。2000 関数でも数十秒)。full=True のときだけ全ページ再レンダし、サイト内検索の索引も更新する
<code>build_pdf</code>	root, kind, title, output	dict	種別ごとの合本 PDF。kind: specs / test-specs / test-results
<code>build_viewer</code>	root, name?, port?, render=True, onedir=False	dict	WBS/仕様書サイト同梱の単体実行ファイル(EXE)を dist/ に生成。1~2 分かかるため呼び出し側のタイムアウトに注意。PyInstaller 必要

実装上の契約

- `run_tests` は verify NG なら `pytest` を実行せず `verify_ng` で返す(stale なテストの pass を防ぐ)
- `subprocess` 系の返り値は `{ok, exit_code, output}`(output は末尾 4000 文字。PYTHONIOENCODING=utf-8 を強制し Windows のコードページ問題を回避)
- `progress_summary` は `ledger status --summary` の JSON をパースして返す (パース不能時は `{ok: false, output}` に落ちる)
- 大規模(2000 関数級)での再開は `progress_summary + next_actions` から。全関数リストをコンテキストに読み込まない(`pipeline_status` の全件取得は分析用途に限る)